

LECTURE-10

Wednesday
30/9/26.

□ Pipelining:

Single Cycle datapath: R-type, I-type (load/store), Branch.

- * 1 clock cycle = max. time taking instruction to be executed
 ↓ this is for "load" instruction which is I-type instruction.
 But since most (80%) of our instructions are R-Type;
 therefore giving upper bound for 1 CC.T wastes times a lot
 & keep CPU idle.

⇒ Instruction Timeline Processor.

- IF Instruction fetch: PC have next instr. address & fetches from ^{memory} instr. ↑
- ID Instruction Decode: Split the opcode to get operand (add, sub)
- EX Execute: ALU performs instruction / operation
- MEM memory access: (load/store)
- WB write Back: register value is written. (load & R type)

□ Multicycle: ^{Opposite of} ↑ pipeline as here the entire instruction timeline is run completely to run the next instruction.

* Pipelining says fetch instructions all the time. Instr. 1 ki "IF" hogai & it moved to "ID"; then we'll put Instr. 2 in the CPU which will do "IF"

Instructions	IF	ID	EX	MEM	WB
1	Ins. 1				
2	2	Ins. 1			
3	3	2	Ins. 1		
4	4	3	2	Ins. 1	
		4	3	2	Ins. 1

add x10, x9, x8

IF: Instruction is read

ID: splits 'add' from registers. x9 & x8 is read for values.
Controls & flags are set

EX: ① ALU perform tasks on value of x9 & x8. **R-type**
② Address & offset is calculated in ALU **Load/store**
③ Subtraction for zero flag to see if to jump **Branch**

MEM: • For R-type; this stage is skipped
• For branching; " " " "
• For load/store; memory is accessed for load or store

WB: value is written back in registers for display ^{for load and} ~~or in~~ **R-type**
~~memory in load/store~~ WB is not needed for store and branch.

NOTE: ① R-type result is available after "EX" stage (not WB)
② load/store result is available after "MEM stage" (not WB)

★ Branching stops after **EX** stage

★ Even any stage is skipped in any type; we'll still include its time.

How much times is pipeline faster = $\frac{\text{Time w/out Pipeline}}{\text{Time with Pipeline}}$

☐ **Forward:** Solution for a problem in Pipelining

① add x10, x9, x8

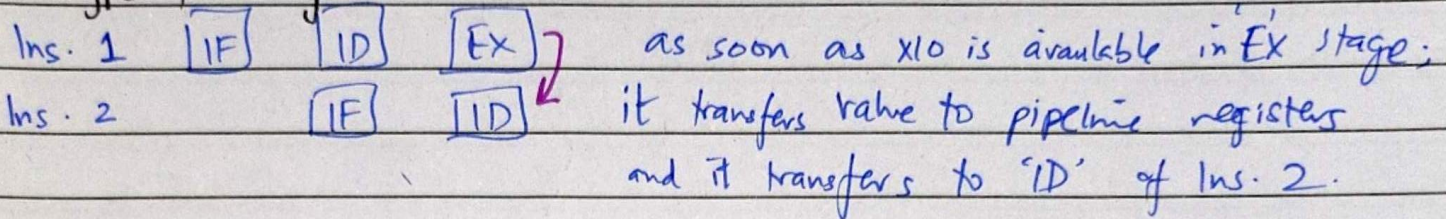
② sub x19, x10, x11

• Here we need the result of ① being written back first completely so ② can use x10 latest result for its processing.
Therefore the CPU needs to stop until ① is fully executed.
This is called **DATA HAZARD**.

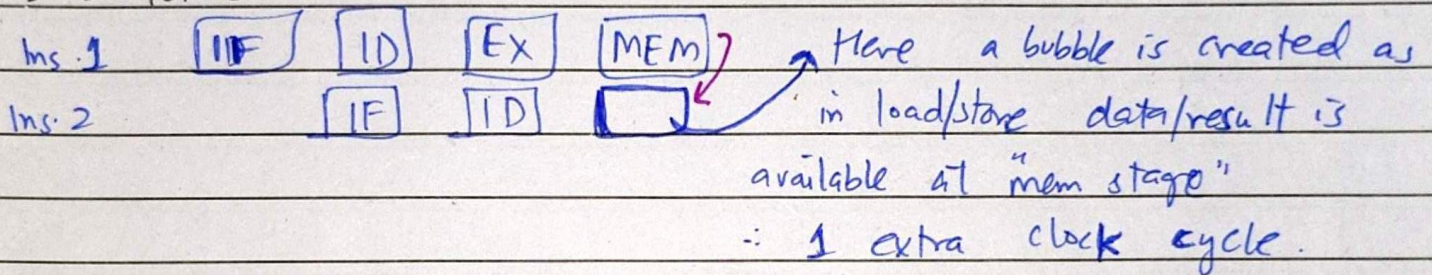
Solution: Use "pipelining registers". This will get the value of x10 as soon as it's available.

So since it's an R-Type instr. $\therefore$ value/result is available after EX stage.

→ R-Type forwarding



→ Load/store



☐ Control Hazard: In Branching

$\left. \begin{array}{l} \text{beg } x1, x2, \text{ loop} \\ \text{add } x3, x4, x5 \end{array} \right\} \text{ Here first the 'beg' needs to be full executed so 'PC' knows whether to jump or not (execute add (next ins)).}$

☐ Structural Hazard: Happens in Multi-Cycle

- When a resource is busy and new instruction or stage comes; it needs to wait
- we removed it in pipelining by designing the machine in such a way.